

Harap Gunakan Format Nama:

NPM_Nama Lengkap



UNIVERSITAS INDONESIA
FAKULTAS ILMU KOMPUTER

Operating System (CSCM602055)

Session 9 : Synchronization Classical Problem and Deadlock

Hendrie Gunawan

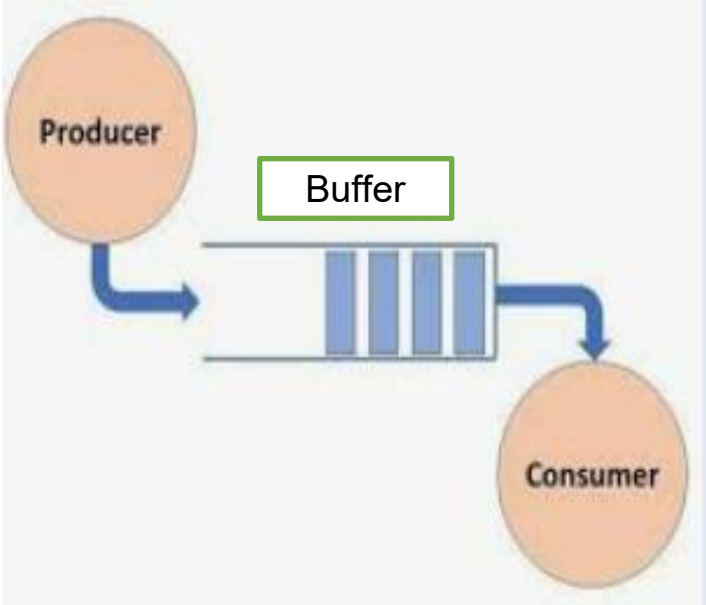
Faculty of Computer Science, University of Indonesia

23 Sept 2026

Objective

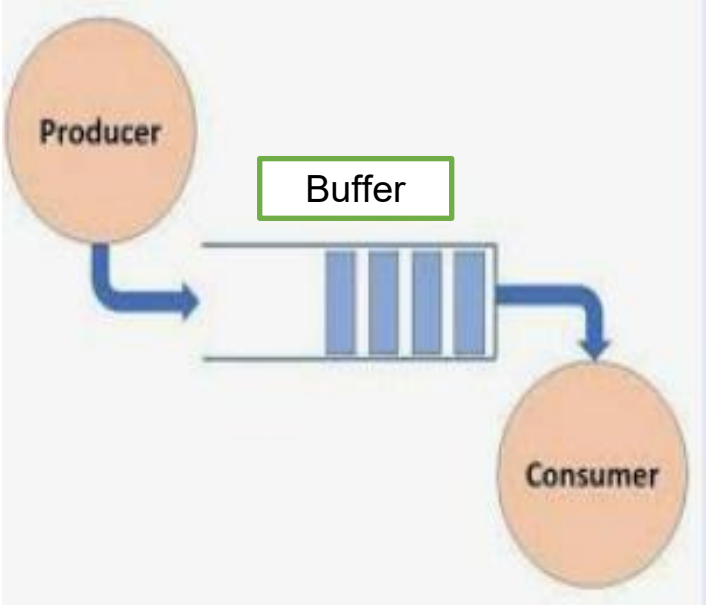
- Mengetahui berbagai Masalah Klasik pada sinkronisasi dan solusinya seperti:
  - Bounded Buffer Problem
  - Readers-Writers Problem
  - Dining-Philosopher Problem
- Memahami definisi dan kondisi-kondisi terjadinya deadlock.
- Mengidentifikasi metode-metode penanganan deadlock (pencegahan, penghindaran, deteksi, dan pemulihan
- Menganalisis algoritma-algoritma terkait deadlock, seperti Banker's Algorithm

1. Bounded Buffer Problem



- Sering disebut **Producer-Consumer** problem
- Ada sebuah buffer sirkular berukuran N.
- Produsen terus-menerus menghasilkan item dan menempatkannya ke dalam buffer.
- Konsumen terus-menerus mengambil item dari buffer dan mengonsumsinya
- Kondisi Penting:
  - Produsen tidak boleh menaruh item jika buffer penuh.
  - Konsumen tidak boleh mengambil item jika buffer kosong.
  - Akses ke buffer (dan variabel indeks in, out, serta count jika digunakan) harus saling eksklusif.

1. Bounded Buffer Problem



- Variabel Bersama:
  - buffer[BUFFER_SIZE]: Array untuk menyimpan item.
  - in = 0: Indeks untuk menambah item (next empty slot).
  - out = 0: Indeks untuk mengambil item (first full slot).
  - Semaphore yang Digunakan:
    - **semaphore mutex = 1**; (Binary semaphore untuk mutual exclusion akses ke buffer itu sendiri: in, out, buffer array.)
    - **semaphore empty = BUFFER_SIZE**; (Counting semaphore, melacak jumlah slot kosong di buffer. Inisialisasi dengan ukuran buffer).
    - **semaphore full = 0**; (Counting semaphore, melacak jumlah slot terisi di buffer. Inisialisasi dengan 0).

1. Bounded Buffer Problem

```
int buffer[MAX]; // tempat penyimpanan datanya
int fill = 0; // tempat Produsen akan menaruh barang
int use = 0; // tempat Konsumen akan mengambil
barang.

void put(int value) { // Produsen ingin menaruh barang
        buffer[fill] = value; // Line F1
        fill = (fill + 1) % MAX; // Line F2
}

int get() { // Konsumen ingin mengambil barang
        int tmp = buffer[use]; // Line G1
        use = (use + 1) % MAX; // Line G2
        return tmp;
}
```

Apa Masalah yang akan terjadi pada Solusi ini ?

1. Jika producer dan consumer berjalan bersamaan :
   - Producer bisa menulis saat consumer membaca.
   - Dua producer bisa menulis di slot sama.
   - Dua consumer bisa membaca data sama.
2. Tidak tahu penuh / kosong

Solusi ini perlu perbaikan dengan cara :

- Mutex/Lock
- Counter atau
- Semaphore
  - **Empty**
  - **Full**
  - **mutex**

Bounded Buffer Problem (semaphore full&empty)

```
sem_t empty;
sem_t full;

void *producer(void *arg) {
        int i;
        for (i = 0; i < loops; i++) {
                sem_wait(&empty); // Line P1
                put(i); // Line P2
                sem_post(&full); // Line P3
        }
}

void *consumer(void *arg) {
        int tmp = 0;
        while (tmp != -1) {
                sem_wait(&full); // Line C1
                tmp = get(); // Line C2
                sem_post(&empty); // Line C3
                printf("%d\n", tmp);
        }
}

int main(int argc, char *argv[]) {
// ...
        sem_init(&empty, 0, MAX); // MAX are empty
        sem_init(&full, 0, 0); // 0 are full
// ...
}
```

- Solusi disamping menambahkan full dan empty semaphore
- Masalah Buffer Overflow pada Producer dan Consumer akan dapat teratasi
- **Akankah Muncul Masalah Lainnya ?**

1. put() dan get() akses variabel shared
2. Jika **lebih dari satu producer** atau **lebih dari satu consumer**, tetap bisa terjadi **Race Condition**
3. Sehingga baik perlu adanya mutex saat access shared variable

Bounded Buffer Problem (full, empty dan mutex)

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++ {
            sem_wait(&mutex); // Kunci dulu pintunya
            sem_wait(&empty); // cek apakah ada slot
kosong
            put(i); // Taruh kopi
            sem_post(&full); // Beri sinyal ada kopi baru
            sem_post(&mutex); // Buka lagi pintunya
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex); // Kunci dulu pintunya
        sem_wait(&full); // Baru cek apakah ada kopi
        int tmp = get(); // Ambil kopi
        sem_post(&empty); // Beri sinyal ada slot kosong
baru
        sem_post(&mutex); // Buka lagi pintunya
        printf("%d\n", tmp);
    }
}
```

- Solusi disamping menambahkan full dan empty semaphore serta mutex untuk menjamin akses ke shared variable
- **Akankah Muncul Masalah Lainnya ?**

1. Perhatikan penempatan mutex, Thread mengunci mutex dulu, baru menunggu resource (empty/full) **Ini pola yang sangat berbahaya**
2. Dapat menyebabkan Consumer menunggu producer isi buffer dan Producer menunggu mutex dilepas consumer.
3. **Keduanya saling tunggu selamanya = DEADLOCK**
4. **Kode Perlu di revisi**

Bounded Buffer Problem (full, empty dan mutex)

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // P1: Cek slot kosong DULU
        sem_wait(&mutex); // P1.5 (lock): Kunci Pintu
        put(i); // P2: Taruh kopi (Critical Section)
        sem_post(&mutex); // P2.5 (unlock): Buka Pintu
        sem_post(&full); // P3: Beri sinyal ada kopi baru
    }
}

void *consumer(void *arg) {
    int i:
    for (i = 0; i < loops; i++) {
        sem_wait(&full); // C1: Cek ada kopi DULU
        sem_wait(&mutex); // C1.5 (lock): Kunci Pintu
        int tmp = get(); // C2: Ambil kopi (Critical Section)
        sem_post(&mutex); // C2.5 (unlock): Buka Pintu
        sem_post(&empty); // C3: Beri sinyal ada slot
kosong
        printf("%d\n", tmp);
    }
}
```

- Solusi adalah perubahan pada kode sebelumnya
- **Akankah Muncul Masalah Lainnya ?**

- Kode disamping sudah benar, kenapa tidak deadlock?
  - Karena thread **tidak pernah block di empty/full sambil memegang mutex**
  - Mutex hanya dipakai singkat saat critical section.
  - menghilangkan circular wait.

✅ **Mutual Exclusion**
Hanya satu thread akses buffer.

✅ **No Buffer Overflow**
Producer tunggu `empty`.

✅ **No Buffer Underflow**
Consumer tunggu `full`.

✅ **No Deadlock**
Urutan semaphore benar.

2. Reader-Writers Problem

- Sumber data tunggal (misalnya, database) diakses oleh banyak proses
  - **Readers**: Proses yang hanya membaca data. Mereka dapat mengakses data secara bersamaan tanpa masalah.
  - **Writers**: Proses yang menulis/memodifikasi data. Mereka harus memiliki akses eksklusif ke data
- Ketentuan Readers-Writers Problem
  - **Banyak Pembaca Boleh Masuk** Bersamaan
  - Hanya **Satu Penulis** yang Boleh Masuk
  - **Penulis** dan **Pembaca Tidak Boleh Bersamaan**

2. Reader-Writers Problem

- **Dua Varian Masalah (Prioritas):**
  - **First Readers-Writers Problem** (Prioritas Reader):
    - Tidak ada reader yang harus menunggu jika ada reader lain yang sedang membaca, kecuali jika seorang writer sudah menulis.
    - Jika seorang writer menunggu, reader baru tidak boleh diizinkan membaca sampai writer selesai. (Namun, ini bisa menyebabkan writer starvation jika reader terus-menerus datang).
  - **Second Readers-Writers Problem** (Prioritas Writer):
    - Jika seorang writer siap, ia harus segera menulis.
    - Reader baru tidak dapat memulai membaca jika ada writer yang menunggu. (Ini bisa menyebabkan reader starvation).

2. Reader-Writers Problem

- **Solusi Menggunakan Semaphore (First Readers-Writers Problem):**
  - semaphore rw_mutex = 1;
    - Binary semaphore untuk mutual exclusion oleh writer atau reader pertama/terakhir.
    - Penulis selalu menunggu rw_mutex.
    - Pembaca pertama wait(rw_mutex) dan pembaca terakhir signal(rw_mutex).
  - semaphore mutex = 1;
    - Binary semaphore untuk mutual exclusion akses ke variabel read_count.
  - int read_count = 0; (Integer, melacak jumlah reader yang sedang membaca).

2. Readers-Writers Problem

```
while (true) {
        wait(rw_mutex);
        ...
        /* writing is performed */
    ...
        signal(rw_mutex);
}

while (true){
        wait(mutex);
        read_count++;
        if (read_count == 1) /* first reader */
                wait(rw_mutex);

        signal(mutex);
    ...
        /* reading is performed */
    ...
        wait(mutex);
        read_count--;
        if (read_count == 0) /* last reader */
                signal(rw_mutex);

        signal(mutex);
}
```

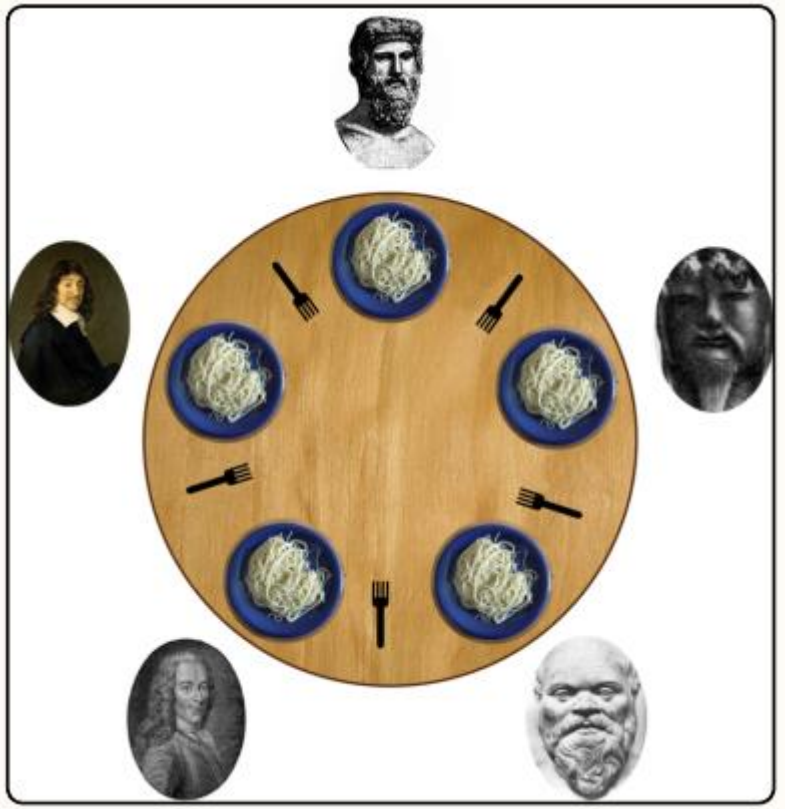
- Kode ini dapat mengakibatkan situasi di mana proses penulis tidak pernah menulis. Ini disebut sebagai masalah “First Reader-Writers Problem".
- Masalah " Second Reader-Writers Problem " adalah variasi dari First Readers-Writers yang menyatakan:
  - Setelah penulis siap untuk menulis, tidak ada "pembaca yang baru datang" yang diizinkan untuk membaca.
- Baik yang pertama maupun yang kedua dapat mengakibatkan **starvation**, yang menyebabkan lebih banyak variasi lagi.
- Masalah ini diatasi pada beberapa sistem dengan kernel menyediakan readers-writers lock.

Solusi Starvation Pada Readers-Writers Problem

1. **Solusi yang Memprioritaskan Penulis (Writer-Preference)**
   - Jika seorang penulis sedang menunggu,jangan izinkan pembaca baru untuk masuk.
2. **Solusi yang Lebih Adil (Menggunakan Antrian)**
   - Siapa datang duluan,dia dilayani duluan, tidak peduli dia pembaca atau penulis.Ini adalah Solusi yang paling adil
   - Solusi ini mengorbankan Sebagian besar keuntungan performa dari pembacaan paralel

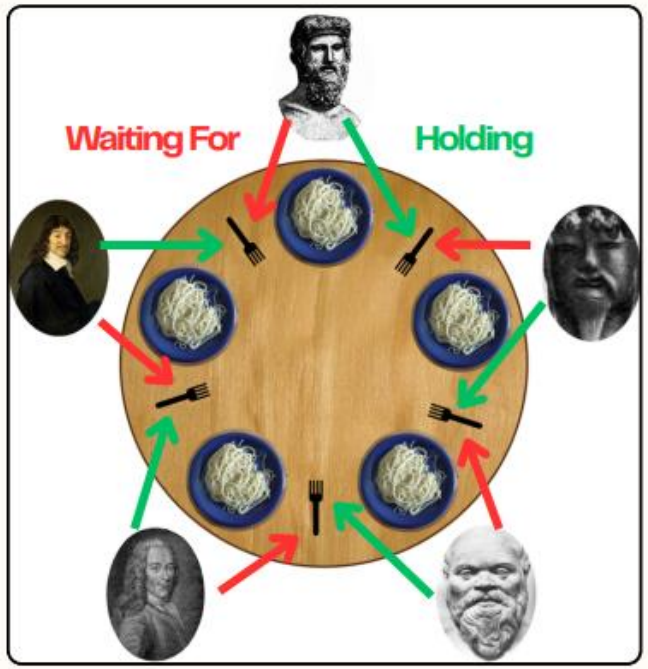
Dining-Philopher Problem

- Masalah Detail:
  - Lima filosof duduk mengelilingi meja bundar.
  - Ada lima sumpit, satu di antara setiap pasangan filosof.
  - Untuk makan, seorang filosof membutuhkan dua sumpit: satu dari kiri dan satu dari kanan.
  - Filosof bergantian antara berpikir dan makan.
- Representasi: Filosof sebagai proses/thread. Sumpit sebagai sumber daya (masing-masing dilindungi oleh sebuah semaphore).
- Kebutuhan Sinkronisasi: Memastikan semua filosof dapat makan (tidak deadlock) dan tidak ada yang mati kelaparan (tidak starvation).



3. Dining-Philopher Problem

```
while (true) {
        pikir();
        wait(garpu[kiri]);
        wait(garpu[kanan]);
        makan();
        signal(garpu[kanan]);
        signal(garpu[kiri]);
}
```



Terjadinya Deadlock

Kasus: Setiap filsuf mengambil garpu kirinya secara bersamaan.

Semua garpu telah dipegang → **Setiap filsuf nunggu garpu kanan.**

↓

Terbentuk siklus tunggu → **Sistem berhenti total.**

3. Dining-Philopher Problem

```
#define N 5

sem_t fork[N]; // tiap garpu = 1 semaphore
sem_t room; // batasi maksimal 4 filsuf mencoba makan

for (int i = 0; i < N; i++)
    sem_init(&fork[i], 0, 1);

sem_init(&room, 0, 4);
```

```
void *philosopher(void *num) {
int i = *(int *)num;

while (1) {
think(i);

sem_wait(&room); // masuk ruang makan

sem_wait(&fork[i]); // ambil kiri
sem_wait(&fork[(i+1)%N]); // ambil kanan

eat(i);

sem_post(&fork[(i+1)%N]); // taruh kanan
sem_post(&fork[i]); // taruh kiri

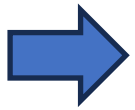
sem_post(&room); // keluar ruang makan
}
}
```

- Batasi hanya **4 filsuf** yang boleh mencoba ambil garpu bersamaan.
- Dengan 5 filsuf:
  - kalau semua 5 masuk → bisa deadlock.
  - kalau hanya 4 masuk → selalu ada 1 garpu bebas → salah satu bisa makan.

✅ No Deadlock

✅ Mudah diimplementasi

✅ Pakai semaphore standar



⚠️ Starvation

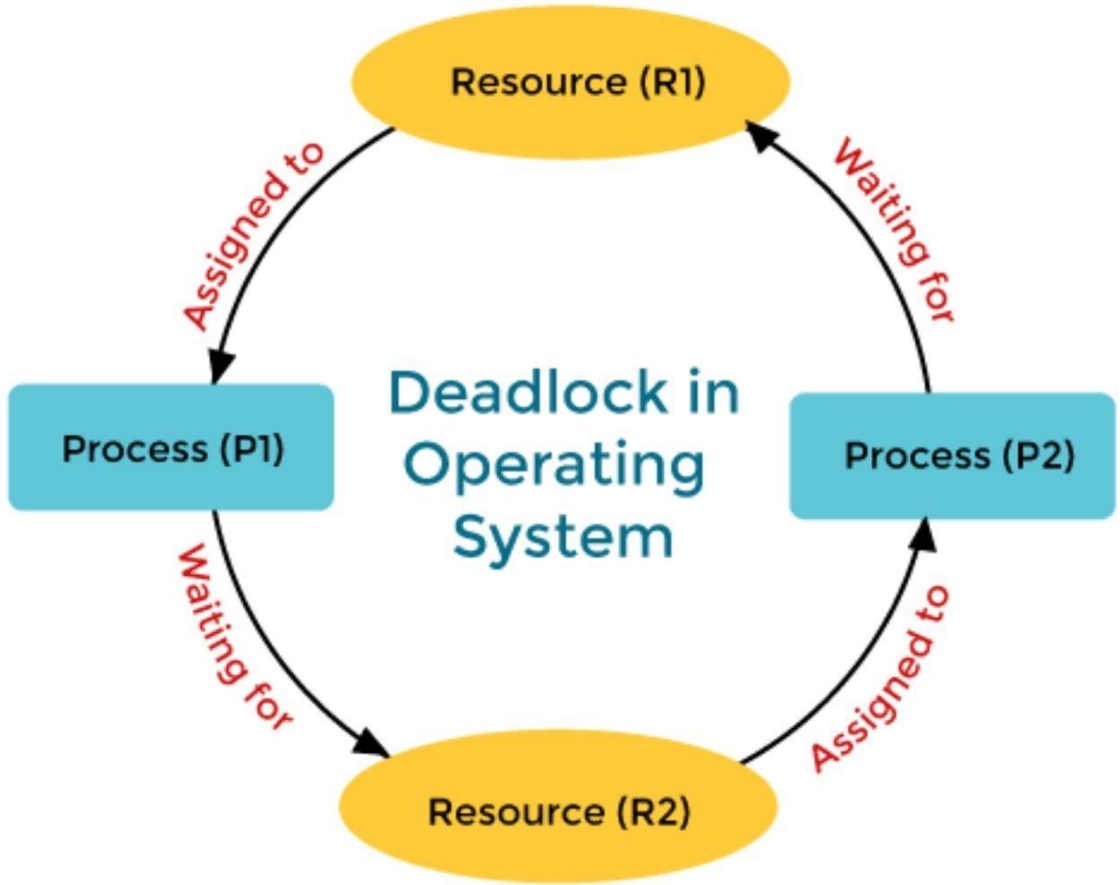
- Filsuf tertentu bisa kalah terus.
- Jika ingin adil → gunakan queue/FIFO semaphore.

Classical Synchronization Problem

- **Solusi Bebas Deadlock**
  - **Mengizinkan Maksimal N-1 Filosof Makan**: Hanya izinkan empat filosof untuk mencoba mengambil sumpit pada satu waktu. Ini menghilangkan kondisi circular wait. (Tambahkan satu counting semaphore global yang diinisialisasi N-1).
  - **Filosof Ganjil/Genap (Asymmetric Solution):** Filosof dengan indeks ganjil mengambil sumpit kiri terlebih dahulu, lalu kanan. Filosof dengan indeks genap mengambil sumpit kanan terlebih dahulu, lalu kiri. Ini memecah circular wait.
  - **Filosof Hanya Makan Jika Kedua Sumpit Tersedia**: Filosof hanya mengambil sumpit jika kedua sumpitnya (kiri dan kanan) tersedia secara bersamaan. Ini dapat dicapai dengan mutex tambahan yang melindungi pengambilan kedua sumpit sebagai sebuah critical section.

Classical Synchronization Problem

- Masalah ini adalah contoh yang sangat baik untuk memahami kondisi deadlock dan bagaimana strategi pencegahan deadlock (melanggar salah satu Coffman Conditions) dapat diimplementasikan
- Coffman Condition :
  - Mutex
  - Hold and Wait
  - No Preemption
  - Circular Wait


Resource (R1)
Assigned to
Waiting for
Process (P1)
Deadlock in Operating System
Process (P2)
Waiting for
Assigned to
Resource (R2)

Model Sistem

- Sistem terdiri dari resources
- Tipe resource seperti R1, R2, …., Rn
  - Terdiri dari CPU Cycle, Memory Space dan I/O Devices
- Setiap resource Ri memiliki Wi instance
- Setiap process, mengutilisasi resource sbb :
  - request
  - use
  - release

Deadlock with Semaphores

- **Data:**
  - A semaphore $\mathbf{s_1}$ initialized to 1
  - A semaphore $\mathbf{s_2}$ initialized to 1
- Two threads T_1 and T_2
- T_1:

  `wait(s1)`

  `wait(s2)`
- T_2:

  `wait(s2)`

  `wait(s1)`

Deadlock

- Situasi di mana **dua atau lebih proses saling menunggu** resource yang dipegang oleh proses lain, sehingga tidak ada proses yang dapat melanjutkan eksekusi
- Analogi kehidupan nyata: Dua mobil atau lebih di persimpangan jalan sempit yang saling memblokir
- Kondisi-Kondisi terjadinya deadlock dalam sistem operasi disebut **Coffman Condition**
- **Coffman condition** ini terdiri dari 4 kondisi dimana keseluruhannya **harus** terpenuhi agar terjadi deadlock

Coffman Condition

1. **Mutual Exclusion (Saling Pengecualian):** Setidaknya satu resource harus berada dalam mode non-shareable, artinya hanya satu proses pada satu waktu yang dapat menggunakan resource tersebut
   - Contoh: Printer, file yang terkunci
2. **Hold and Wait (Menahan dan Menunggu):** Proses memegang setidaknya satu resource dan menunggu untuk memperoleh resource tambahan yang saat ini dipegang oleh proses lain
   - Contoh: Proses A memegang Printer A dan menunggu Printer B yang dipegang oleh Proses B

Coffman Condition (2)

3. **No Preemption (Tidak Ada Preemptif):** resource tidak dapat diambil secara paksa dari proses yang memegangnya; resource harus dilepaskan secara sukarela oleh proses setelah selesai menggunakannya
   - Contoh: Anda tidak bisa secara paksa mengambil buku dari seseorang yang sedang membacanya
4. **Circular Wait (Menunggu Melingkar):** Terdapat serangkaian proses $P_0,P_1,\ldots,P_n$ sedemikian rupa sehingga P_0 menunggu resource yang dipegang oleh P_1,P_1 menunggu resource yang dipegang oleh $P_2,\ldots,P_{n-1}$ menunggu resource yang dipegang oleh P_n, dan P_n menunggu resource yang dipegang oleh P_0

4 Metode Penanganan Deadlock

1. Pencegahan Deadlock (Deadlock Prevention)
2. Penghindaran Deadlock (Deadlock Avoidance)
3. Pendeteksian Deadlock (Deadlock Detection)
4. Pemulihan Deadlock (Deadlock Recovery)

1. Pencegahan Deadlock (Deadlock Prevention)

- Bertujuan untuk memastikan bahwa setidaknya satu dari empat kondisi Coffman tidak pernah terpenuhi
  - **Mencegah Mutual Exclusion:**
    - Jarang mungkin, karena beberapa resource memang secara intrinsik non-shareable (misalnya, printer). Dapat diatasi dengan spooling
  - **Mencegah Hold and Wait:**
    - Proses harus meminta semua resource yang dibutuhkannya sebelum memulai eksekusi
    - Proses harus melepaskan semua resource yang dipegangnya sebelum meminta resource baru
    - Kerugian: Utilasi resource rendah, starvasi (proses mungkin harus menunggu lama untuk semua resource)

1. Pencegahan Deadlock (Deadlock Prevention)

- **Mencegah No Preemption**
  - Jika suatu proses meminta resource yang tidak tersedia, dan ia memegang resource lain, maka semua resource yang dipegangnya dapat diambil paksa
  - Memungkinkan jika state proses dapat dengan mudah disimpan dan dipulihkan (misalnya, CPU state). Sulit untuk resource seperti printer
- **Mencegah Circular Wait**
  - Menetapkan urutan numerik global untuk semua tipe resource
  - Proses hanya dapat meminta resource dalam urutan naik
    - Contoh: resource R1, R2, R3. Proses harus meminta R1 sebelum R2, dan R2 sebelum R3
  - Kerugian: Mungkin tidak selalu praktis atau efisien, pembatasan fleksibilitas

1. Pencegahan Deadlock (Deadlock Prevention)

- **Mencegah Circular Wait**
  - Membatalkan kondisi tunggu melingkar adalah hal yang paling umum.
  - Tetapkan saja setiap resource (misalnya mutex lock) dengan nomor yang unik.
  - Resources must be acquired in order.
  - If:
    first_mutex = 1
    second_mutex = 5

    kode untuk thread_two tidak dapat ditulis sebagai berikut :

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

2. Penghindaran Deadlock (Deadlock Avoidance)

- Sistem secara dinamis memutuskan apakah alokasi resource saat ini akan menyebabkan deadlock
- Membutuhkan informasi tambahan tentang resource apa yang akan diminta dan dilepaskan oleh setiap proses
- Sistem hanya akan mengalokasikan resource jika hasilnya adalah state yang "aman" (safe state)
  - **Safe State** adalah Keadaan di mana ada urutan alokasi resource untuk semua proses yang memungkinkan setiap proses untuk menyelesaikan eksekusinya

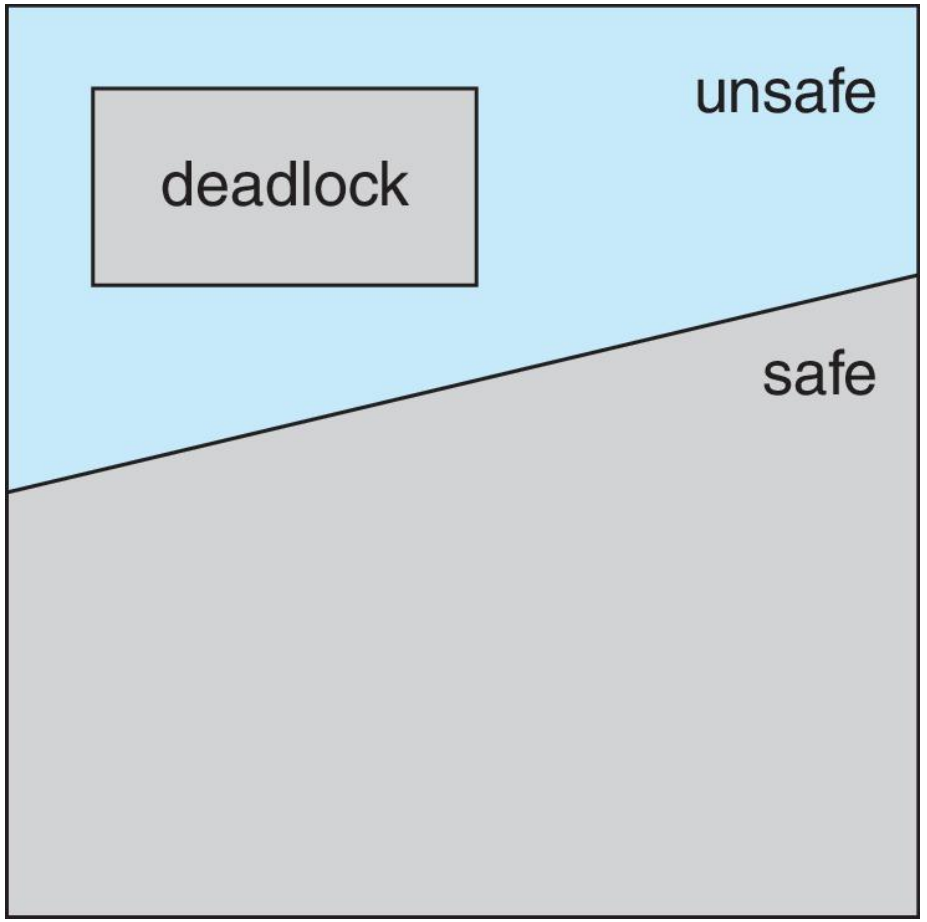
Safe State

- Ketika sebuah thread meminta resource yang tersedia, sistem harus memutuskan apakah alokasi segera membuat sistem dalam status aman
- Sistem berada dalam safe state jika terdapat urutan <T1, T2, …, Tn> dari SEMUA thread dalam sistem sehingga untuk setiap Ti, resource yang masih dapat diminta Ti dapat dipenuhi oleh resource yang tersedia saat ini + resource yang dimiliki oleh semua Tj, dengan j < I yaitu :
  - Jika kebutuhan resource Ti tidak segera tersedia, maka Ti dapat menunggu hingga semua Tj selesai
  - Ketika Tj selesai, Ti dapat memperoleh resource yang dibutuhkan, mengeksekusi, mengembalikan resource yang dialokasikan, dan mengakhiri
  - Ketika Ti mengakhiri, Ti +1 dapat memperoleh resource yang dibutuhkannya, dan seterusnya

Basic Fact

- Jika sistem dalam keadaan aman ➔ tidak ada deadlock
- Jika sistem dalam keadaan tidak aman ➔ kemungkinan deadlock
- Penghindaran (avoidance) ➔ memastikan bahwa sistem tidak akan pernah memasuki keadaan tidak aman

Safe, Unsafe, Deadlock State



Algoritma Deadlock Avoidance

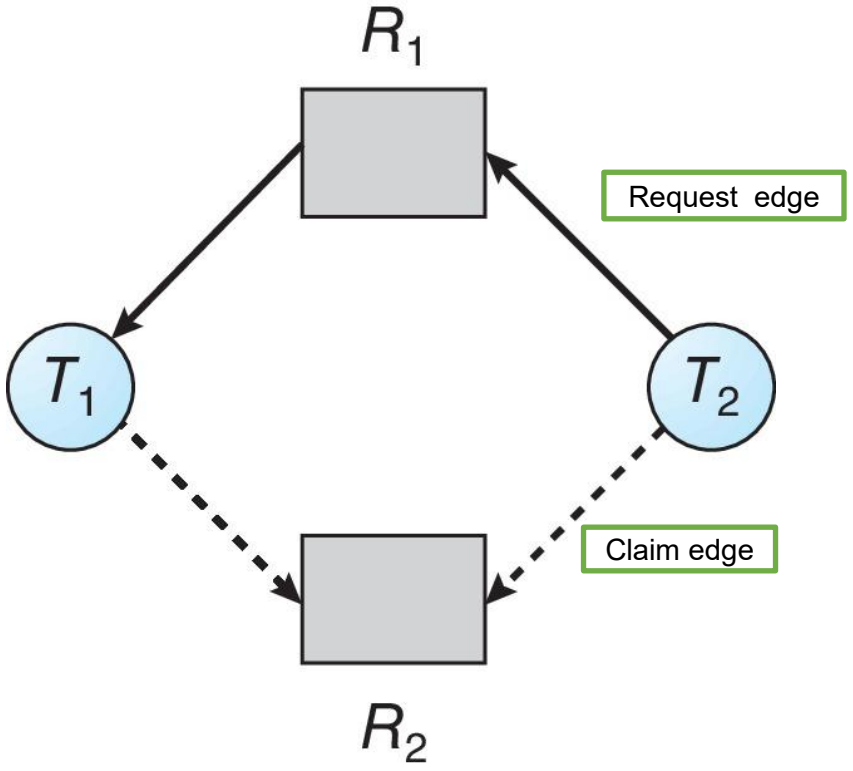
- **Algoritma Graf Alokasi resource (Resource-Allocation Graph Algorithm):**
  - Digunakan untuk **single instance** of each resource type (satu contoh setiap tipe resource). Jika ada siklus, berpotensi deadlock
- **Banker's Algorithm:**
  - Digunakan untuk **multiple instances** of each resource type (beberapa contoh setiap tipe resource)
  - Membutuhkan setiap proses untuk mendeklarasikan jumlah maksimum resource dari setiap tipe yang mungkin dibutuhkannya

Resource Allocation Graph

- Sebuah set yang terdiri dari vector *V* dan sisi-sisi *E,* maka
  - V dipartisi menjadi 2 bentuk:
    - $T = \{T_1, T_2, \ldots, T_n\}$, set yang terdiri dari seluruh thread dalam sistem.
    - $R = \{R_1, R_2, \ldots, R_m\}$, set yang terdiri dari seluruh tipe resource didalam sistem
- **request edge** – directed edge $T_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow T_i$

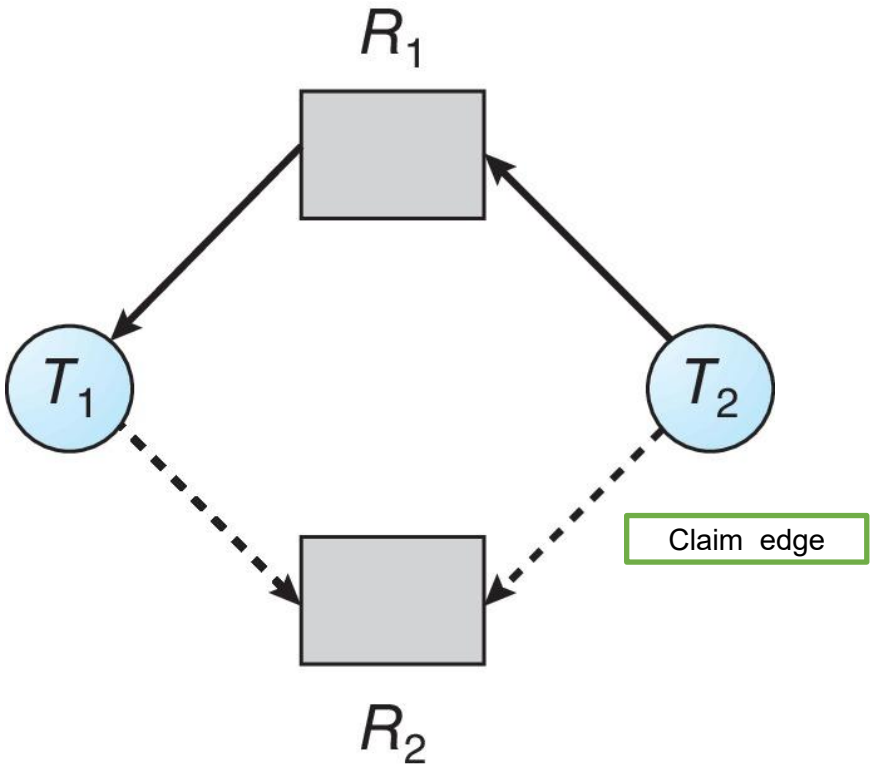
Resource Allocation Graph

- Claim Edge T_i -> R_j menunjukkan bahwa proses T_j dapat meminta resource R_j; diwakili oleh garis putus-putus
- Claim Edge diubah menjadi request edge saat thread meminta resource
- Request edge diubah menjadi assignment edge saat resource dialokasikan ke thread
- Saat resource dilepaskan oleh thread, assignment edge diubah lagi menjadi claim edge
- resource harus diklaim terlebih dahulu (priori) dalam sistem



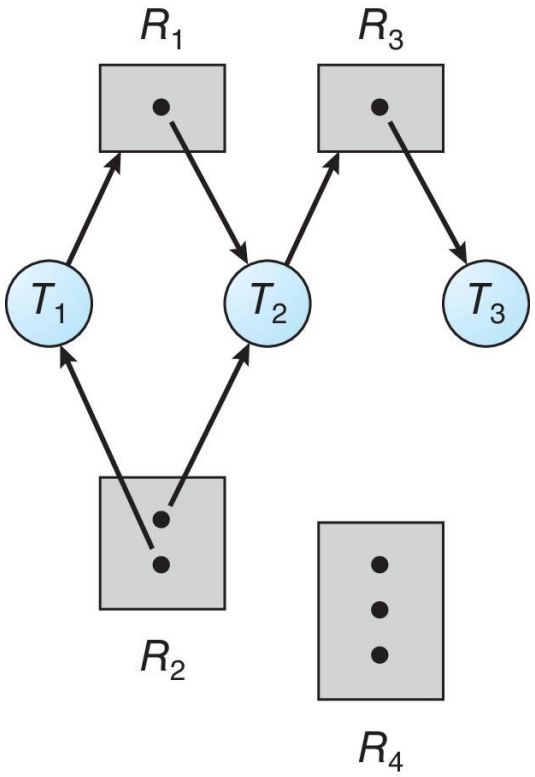
Resource Allocation Graph Algorithm

- Misalkan thread T_i meminta resource R_j
- Permintaan dapat dikabulkan hanya jika mengubah tepi permintaan menjadi tepi penugasan tidak mengakibatkan pembentukan siklus dalam grafik alokasi resource.



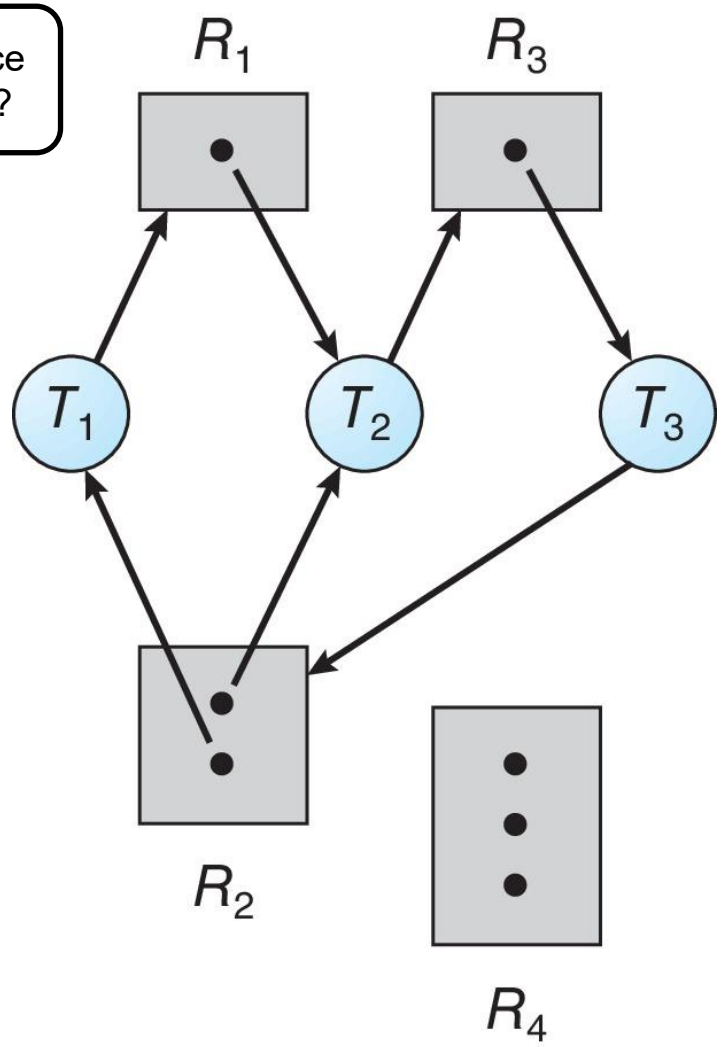
Resource Allocation Graph Sample

- Satu instance R_1
- Dua instance R_2
- Satu instance R_3
- Tiga instance R_4
- T_1 memegang satu instance dari R_2 and menunggu instance R_1
- T_2 memegang satu instance dari R_1, satu instance dari R_2, dan menunggu instance dari R_3
- T_3 memegang instance R_3



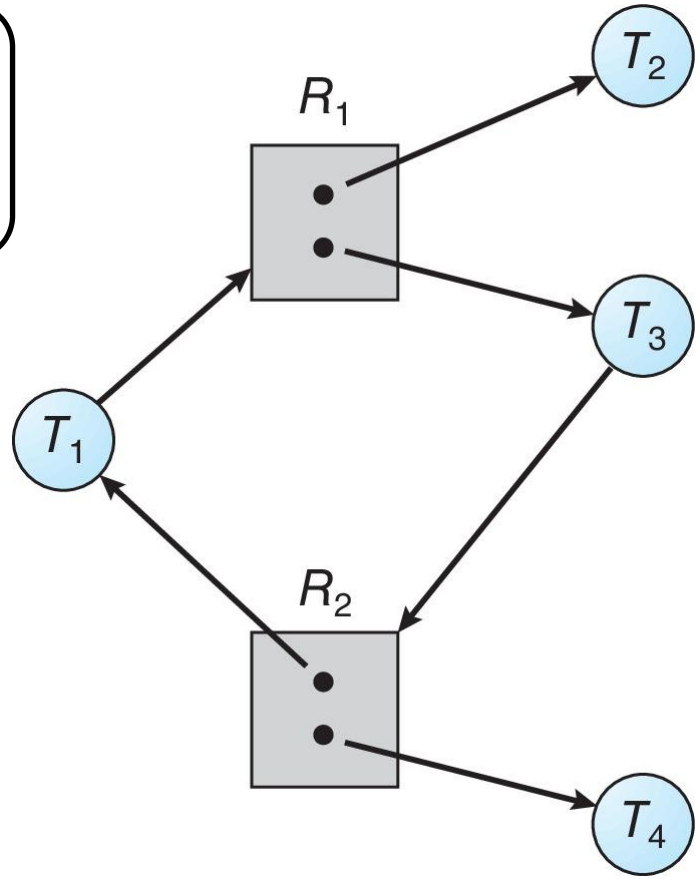
Resource Allocation Graph with a Deadlock

Bagaimana Jika T3 Meminta Resource dari R2 dan menunggu instance R2 ?



Terjadi Deadlock karena adanya Circular Wait

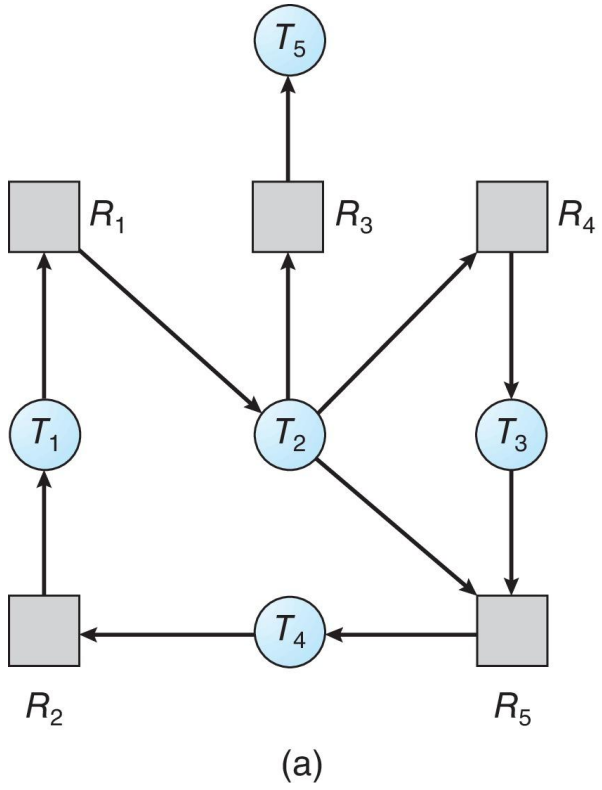
Bagaimana dengan Ini?



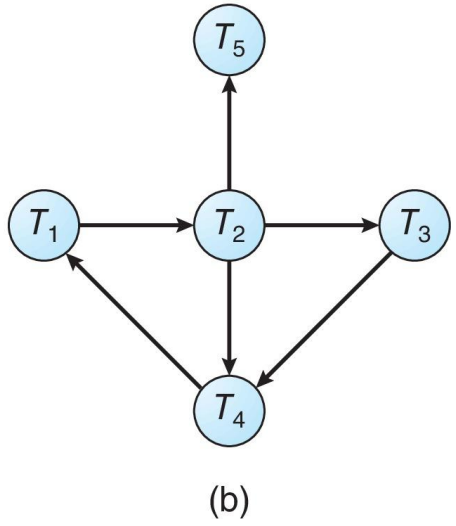
Tidak Terjadi Deadlock Walaupun ada Circular Wait Karena T4 dapat menyelesaikan threadnya

Single Instance of Each Resource Type

- Maintain **wait-for** graph
  - Nodes adalah threads
  - $T_i \rightarrow T_j$ jika T_i menunggu T_j
- Secara berkala panggil algoritma yang mencari siklus dalam grafik. Jika ada siklus, berarti ada deadlock
- Algoritma untuk mendeteksi siklus pada grafik memerlukan urutan operasi n^2, dimana n adalah jumlah titik sudut pada grafik.



Resource-Allocation Graph



Corresponding wait-for graph

Fakta

- Jika grafik tidak mengandung siklus (circular) ➔ tidak ada deadlock
- Jika grafik mengandung siklus ➔
  - Jika hanya satu instance per tipe resource, maka deadlock
  - Jika beberapa instance per tipe resource, kemungkinan deadlock

Banker's Algorithm

- **Initial State:** Sistem memeriksa apakah ada cukup resource yang tersedia untuk memenuhi kebutuhan maksimum yang dinyatakan oleh semua proses
- Let *n* = number of processes, and *m* = number of resources types
  - **Available***:* Vector of length *m*. If available [*j*] = *k*, there are *k* instances of resource type R_j available
  - **Max***: n x m* matrix. If *Max* [*i,j*] = *k*, then process T_i may request at most *k* instances of resource type R_j
  - **Allocation***: n* x *m* matrix. If Allocation[*i,j*] = *k* then T_i is currently allocated *k* instances of R_j
  - **Need***: n* x *m* matrix. If *Need*[*i,j*] = *k*, then T_i may need *k* more instances of R_j to complete its task
  - *Need* [*i,j*] = *Max*[*i,j*] – *Allocation* [*i,j*]

Banker's Algorithm

- **Safety Algorithm :** Menentukan apakah sistem berada dalam safe state
  1. **Work**: Vektor panjang m (jumlah tipe resource) yang menunjukkan resource yang tersedia.
  2. **Finish**: Vektor boolean panjang n (jumlah proses) yang menunjukkan apakah proses telah selesai.
  3. **Temukan i sedemikian rupa sehingga Finish[i] == false dan Need[i] <= Work. Jika tidak ada i seperti itu, goto langkah 4.**
  4. **Work = Work + Allocation[i].**
  5. **Finish[i] = true.** Goto langkah 2**.**
  6. **Jika Finish[i] == true untuk semua i,** maka sistem berada dalam safe state.

Contoh Banker's Algorithm

- 5 threads T_0 sampai T_4;
  3 resource types:
  A (10 instances), *B* (5 instances), and *C* (7 instances)
- Snapshot pada saat T_0:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

Example (Cont.)

- Konten dari matrix ***Need*** didefinisikan sebagai ***Max – Allocation***
- **Apakah sistem dalam keadaan safe state dengan sequence** sequence < T_1, T_3, T_4, T_2, T_0>?

	Allocation	Max	Available	*Need*
	A B C	*A B C*	*A B C*	*A B C*
T_0	0 1 0	7 5 3	3 3 2	7 4 3
T_1	2 0 0	3 2 2		1 2 2
T_2	3 0 2	9 0 2		6 0 0
T_3	2 1 1	2 2 2		0 1 1
T_4	0 0 2	4 3 3		4 3 1

Example (Cont.)

Step	Process	Available Sebelum	Available Sesudah
1	T1	(3,3,2)	(5,3,2)
2	T3	(5,3,2)	(7,4,3)
3	T4	(7,4,3)	(7,4,5)
4	T2	(7,4,5)	(10,4,7)
5	T0	(10,4,7)	(10,5,7)

- Sistem dalam safe state karena sequence < T_1, T_3, T_4, T_2, T_0> memenuhi kriteria safety

3. Deteksi Deadlock (Deadlock Detection)

- Memungkinkan sistem memasuki state deadlock, lalu mendeteksi dan memulihkannya
- Cocok untuk situasi di mana frekuensi deadlock rendah dan overhead pencegahan/penghindaran terlalu tinggi
- **Algoritma Deteksi (Mirip dengan Banker's Safety Algorithm)**

3. Deteksi Deadlock (Deadlock Detection)

- **Algoritma Deteksi (Mirip dengan Banker's Safety Algorithm)**
  - **Available**: Vektor resource yang tersedia.
  - **Allocation**: Matriks resource yang dialokasikan ke setiap proses.
  - **Request**: Matriks resource yang diminta oleh setiap proses.
  - **Langkah-langkah:**
    1. **Work = Available**, Finish[i] = false untuk semua i.
    2. **Temukan i sedemikian rupa sehingga Finish[i] == false dan Request[i] <= Work.** Jika tidak ada i seperti itu, goto langkah 4.
    3. **Work = Work + Allocation[i], Finish[i] = true.** Goto langkah 2.
    4. **Jika Finish[i] == false untuk beberapa i, maka ada deadlock.** Proses yang Finish[i] == false adalah bagian dari siklus deadlock.

3. Deteksi Deadlock (Deadlock Detection)

- Kapan menjalankan algoritma deteksi?
  - Setiap kali permintaan resource dibuat
  - Secara periodic
  - Ketika pemanfaatan CPU menurun (indikasi potensi deadlock)

Detection Algorithm

1. Work dan Finish adalah vektor dengan panjang m dan n, masing-masing di Inisialisasi :
   Work* = *Available
2. For ***i* = 1,2, …, *n***, if ***Allocation*$_i$ ≠ 0**, then
   Finish*[i] = *false; otherwise, ***Finish*[i] = *true***
3. Temukan index ***i*** *sehingga* kedua kondisi berikut terpenuhi:
   a) ***Finish*[*i*] == *false***
   b) ***Request*$_i$ ≤ *Work***
      Jika tidak ada I yang memenuhi maka lanjut ke Langkah 4
4. ***Work* = *Work* + *Allocation*$_i$**
   Finish*[*i*] = *true
   go to step 2
5. If ***Finish[i] == false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if ***Finish*[*i*] == *false***, then T_i is deadlocked

Algorithm requires an order of O(m x n^2) operations to detect whether the system is in deadlocked state

Example of Detection Algorithm

- 5 threads T_0 sampe T_4; dengan 3 tipe resource terdiri dari A (7 instances), *B* (2 instances), and *C* (6 instances)
- Snapshot at time T_0:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	
T_2	3 0 3	0 0 0	
T_3	2 1 1	1 0 0	
T_4	0 0 2	0 0 2	

- sekuens <T_0, T_2, T_3, T_1, T_4> akan menghasilkan ***Finish[i] = true*** untuk semua ***i***

Example (Cont.)

- T_2 meminta tambahan instance dengan tipe ***C***

	Request
	A B C
T_0	0 0 0
T_1	2 0 2
T_2	0 0 1
T_3	1 0 0
T_4	0 0 2

- State of system?
  - Can reclaim resources held by thread T_0, but insufficient resources to fulfill other processes; requests
  - Deadlock exists, consisting of processes T_1, T_2, T_3, and T_4

Detection-Algorithm Usage

- **Kapan, Seberapa sering tergantung pada :**
  - Seberapa sering kebuntuan kemungkinan terjadi?
  - Berapa banyak proses yang perlu dibatalkan?
    - satu untuk setiap siklus terpisah
- Jika algoritma deteksi dipanggil secara sembarangan, mungkin ada banyak siklus dalam grafik resource dan karenanya kita tidak akan dapat mengetahui utas mana dari sekian banyak utas yang menemui jalan buntu yang “menyebabkan” kebuntuan tersebut.

4. Pemulihan Deadlock (Deadlock Recovery)

- Setelah deadlock terdeteksi, sistem harus memulihkannya
- **Metode Pemulihan:**
  - **Terminasi Proses:**
    - Terminasi semua proses yang terlibat dalam deadlock. (Kerugian: kehilangan pekerjaan).
    - Terminasi satu per satu proses yang terlibat sampai siklus deadlock terputus. (Strategi: terminasi proses dengan resource paling sedikit, proses yang paling sedikit progresnya, dll.).
  - **Preemptif resource (Resource Preemption):**
    - Memilih "korban" (victims): resource yang akan diambil dari proses yang terlibat dalam deadlock.
    - **Rollback**: Mengembalikan state proses yang preempted ke state sebelumnya.
    - **Starvation**: Memastikan bahwa proses yang sama tidak selalu menjadi korban.

Dalam urutan apa kita harus memilih untuk membatalkan?

- Prioritas thread
- Berapa lama thread telah dihitung, dan berapa lama lagi hingga selesai
- resource yang telah digunakan thread
- resource yang perlu diselesaikan thread
- Berapa banyak thread yang perlu dihentikan
- Apakah thread bersifat interaktif atau batch?


Apa ada pertanyaan?


JANGAN SUSAH" PERTANYAANMU